

pylablib.devices.Attocube.anc350 コマンドリファレンス

本資料は、Attocube ANC350 コントローラーを制御するための ANC350 クラスおよび関連モジュールにおける主要なメソッドとクラスをまとめたものです。

1. 接続・デバイス情報管理

デバイスの初期化や接続確認、ハードウェア情報の取得に関するコマンドです。

- **get_usb_devices_number()**

USB 経由で接続されているコントローラーの数を取得します。

- **is_connected(axis='all')**

指定した軸（または全軸）が接続されているかを確認します。

- **get_hardware_id()**

デバイスのハードウェア ID を取得します（デフォルトは -1）。

- **set_hardware_id(hwid, persist=False)**

デバイスのハードウェア ID を設定します。persist=True で電源の再起動後も値を保持します。

- **open() / close() / is_opened()**

デバイスバックエンドとの通信を開く、閉じる、または開いているかを確認します。

- **lock(timeout=None) / unlock() / locking(timeout=None)**

他のスレッドやプロセスからのデバイスへのアクセスをロック・アンロックします (locking はコンテキストマネージャーとして使用)。

2. 軸のステータス・情報取得

各軸の状態、リミット、静電容量などを確認するコマンドです。

- **get_status(axis='all')**

デバイスのステータス文字列を取得します（例: running, limit, sens_err など）。

- **get_status_n(axis='all')**

軸の数値ステータスを取得します。

- **check_limit(axis='all')**

移動限界に達しているかを確認します (None, fwd, bwd, both を返します)。

- **get_capacitance(axis='all', measure=False, delay=0.5)**

軸の静電容量（ファラド）を取得します。measure=True で新規に測定を行います。

- **get_position(axis='all')**

軸の現在の位置（メートル）を取得します。

- **get_target_position(axis='all')**

軸が向かっているターゲット位置を取得します。

3. 軸の設定（電圧・周波数・精度）

軸の動作パラメータやセンサーに関する設定を行うコマンドです。

- **enable_axis(axis='all', enabled=True) / disable_axis(axis='all')**

特定の軸、または全軸を有効化・無効化します。

- **is_enabled(axis='all')**

軸が有効になっているかを確認します。

- **set_voltage(axis, voltage) / get_voltage(axis='all')**

軸のステップ電圧（ボルト）を設定・取得します。

- **set_frequency(axis, freq) / get_frequency(axis='all')**

軸のステップ周波数（ヘルツ）を設定・取得します。

- **set_offset(axis, voltage) / get_offset(axis='all')**

軸のオフセット電圧（ボルト）を設定・取得します。

- **set_sensor_voltage(voltage) / get_sensor_voltage()**

位置センサーの電圧（ボルト）を設定・取得します。

- **set_precision(axis='all', precision=1e-06) / get_precision(axis='all')**

ターゲット到達判定に使用する軸の精度（メートル）を設定・取得します。

4. 動作制御（移動・待機・停止）

実際に軸を動かしたり、その動作を制御・監視するコマンドです。

- **move_to(axis, position, precision=None)**

指定したターゲット位置（メートル）へ軸を移動させます。

- **move_by(axis, dist)**

指定した距離（メートル）だけ軸を相対移動させます。

- **move_by_steps(axis, steps=1, delay=0)**

指定したステップ数だけ軸を移動させます。

- **jog(axis, direction)**

指定方向（+ または -）へ軸をジョグ移動させます。明示的に停止するカリミットに達するまで動作が続きます。

- **stop(axis='all')**

指定した軸の移動を停止します。

- **is_moving(axis='all')**

指定した軸が現在移動中であるかを確認します。

- **is_target_reached(axis='all', precision=None)**

ターゲット位置に到達したかを確認します。

- **wait_move(axis, precision=1e-06, timeout=10.0, period=0.01)**

軸の移動が完了する、またはターゲット位置に到達するまで待機します。

5. 低レベル通信・その他設定

より詳細なパラメータ設定や、通信内容を直接操作するためのコマンドです。

- **set_value(address, index, value, ack=False) / get_value(address, index, as_int=True)**

指定したアドレスとインデックスに対して、デバイスの値を直接設定・取得します。

- **enable_updates(enabled=True) / check_tell(timeout=0.01)**

定期的な TELL（値の更新）コマンドの有効化と、キューにある TELL コマンドの確認を行います。

- **apply_settings(settings) / get_settings(include=0)**

デバイスの設定を辞書形式で適用・取得します。

- **get_full_info(include=0) / get_full_status(include=0)**

ステータスや設定を含む詳細な情報を辞書形式で取得します。

- **get_device_variable(key) / set_device_variable(key, value)**

特定の設定パラメータの値を個別に設定・取得します。

- **get_all_axes() / remap_axes(mapping, accept_original=True)**

利用可能な全軸のリスト取得や、軸のラベル（名前）をリマッピングします。

6. 関連クラスと例外（pylablib.devices.Attocube.base 等）

データ構造やエラーハンドリングに使用されるクラス群です。

- **Telegram / Reply**

通信データ（オペコード、アドレス、インデックス、データなど）を格納するためのデータクラス（タプル）です。

- **AttocubeError**

Attocube デバイスに関連する一般的なエラーを表す例外クラスです。

- **AttocubeBackendError**

Attocube のバックエンド通信に関連するエラーを表す例外クラスです。